8086 PUSH and POP Instructions

1. Introduction to Stack

Concept

The **stack** is a special memory area that follows the **LIFO (Last In, First Out)** principle. It is used for:

- Temporary data storage
- Procedure calls and returns
- Saving registers and flags

Rules / Notes

Stack Characteristics

- Stack segment is pointed by **SS**
- Top of stack is pointed by **SP**
- Stack grows **downward** (SP decreases on PUSH)

2. PUSH Instruction

Concept

PUSH transfers **16-bit data** from a source operand to the stack.

Rules / Notes

General Operation

1. SP ← SP - 2
2. Store low and high bytes at SS:SP

Rules / Notes

Valid Sources for PUSH

- 16-bit general purpose registers
- Segment registers
- Immediate data
- 16-bit memory operand
- Flags register
- All general-purpose registers (PUSHA)

2.1 PUSH Register

Example

```
PUSH AX
```

Internal Operation:

$$SS : [SP - 1] \leftarrow AH$$
$$SS : [SP - 2] \leftarrow AL$$
$$SP \leftarrow SP - 2$$

2.2 PUSH Memory

Example

```
PUSH WORD PTR [2000H]
```

Explanation: The 16-bit value stored at `DS:2000H` is pushed onto the stack.

2.3 PUSH Immediate

Example

```
PUSH 1234H
```

Explanation: Immediate 16-bit data is directly pushed onto the stack.

2.4 PUSH Segment Register

Example

```
PUSH DS
PUSH CS
```

Explanation: The contents of the segment register are pushed as 16-bit values.

2.5 PUSHF (Push Flags)

Concept

PUSHF copies the contents of the **FLAGS register** onto the stack.

Example

```
PUSHF
```

2.6 PUSHA (Push All Registers)

Concept

PUSHA pushes all **general-purpose registers** onto the stack.

Rules / Notes

Registers Pushed (Order)

1. AX
2. CX
3. DX
4. BX
5. SP (original value)
6. BP
7. SI
8. DI

Rules / Notes

Important Notes

- Segment registers, IP, FLAGS are **not included**
- Stack space used = 16 bytes
- SP ← SP - 16

Example

```
PUSHA
```

3. POP Instruction

Concept

POP removes 16-bit data from the stack and places it into the destination operand.

Rules / Notes

General Operation

1. Read data from SS:SP
2. SP ← SP + 2

Rules / Notes

Valid Destinations for POP

- 16-bit general purpose registers
- Segment registers
- 16-bit memory locations
- FLAGS register
- All general-purpose registers (POPA)

3.1 POP Register

Example

```
POP BX
```

Internal Operation:

$$BL \leftarrow SS : [SP]$$
$$BH \leftarrow SS : [SP + 1]$$
$$SP \leftarrow SP + 2$$

3.2 POP Memory

Example

```
POP WORD PTR [3000H]
```

Explanation: The word popped from stack is stored at `DS:3000H`.

3.3 POPF (Pop Flags)

Concept

POPF removes a 16-bit value from stack and loads it into the **FLAGS register**.

Example

```
POPF
```

3.4 POPA (Pop All Registers)

Concept

POPA restores all general-purpose registers from the stack.

Rules / Notes

Registers Restored (Order)

1. DI
2. SI
3. BP
4. (SP discarded)
5. BX
6. DX
7. CX
8. AX

Rules / Notes

Important Notes

- Value popped for SP is discarded
- Segment registers are not affected

Example

```
POPA
```

Quick Revision (One Look)

- PUSH → SP decreases by 2
- POP → SP increases by 2
- PUSHA → pushes 8 registers (16 bytes)
- POPA → restores 8 registers (SP ignored)
- PUSHF / POPF → save and restore FLAGS
- Stack grows downward in memory

Exam Tips

Exam Rule:

- PUSH stores data first, then updates SP
- POP retrieves data, then updates SP